



Lab 2.1: C Libraries, Linking, and ELF Executable Structure

Aaisma Wagle

Bsc (Hons) Computer Science with Artificial Intelligence

Softwarica College of IT and E-commerce

ST5003CMD Operating Systems, Security, and Networks

Rupak Rajbanshi

3rd October, 2026

TABLE OF CONTENT

INTRODUCTION	1
A PRACTICAL C PROGRAM: PROCESS INFORMATION	2
A. Create procinfo.c	2
B. Compile procinfo.c	3
THE C STANDARD LIBRARY: TWO HALVES	3
A. Part 1: Header Files: The Construction Manual	3
B. Part 2: Precompiled Machine Code: The Actual Libraries	4
C. Summary: The Two-Part Anatomy	4
STATIC VS. DYNAMIC LINKING	5
A. Static Linking	5
B. Dynamic Linking	5
C. Why It Matters	6
INSPECTING EXECUTABLES WITH READELF	6
A. Three Views of an Executable	6
B. View 1: The ELF Header (readelf -h)	7
C. View 2: The Program Headers (readelf -l)	7
D. View 3: The Dynamic Section (readelf -d)	8
TRACING DEPENDENCIES WITH LDD	9
A. Static Binaries and ldd	9
KNOWLEDGE CHECK QUESTIONS	10
Question 1: What is the fundamental difference between a C header file and a shared library?	10
Question 2: What is static and dynamic linking?	10
Question 3: What is the ELF Header? Write down the commands to check headers.	10
Question 4: What does bookmark mean? Provide an example.	10
CONCLUSION	11
GITHUB REPOSITORY	11
REFERENCES	11

LIST OF FIGURES

Figure 01: Make Lab2.1 Folder to Work With	1
Figure 02: Make procinfo.c File	2
Figure 03: procinfo.c File in Nano Editor	2
Figure 04: procinfo.c File View using cat Command	3
Figure 05: Run GCC compilation with result	3
Figure 06: Header File Check Using ls and grep	3
Figure 07: Checking Shared and Static libc Files	4
Figure 08: Static Compilation and Size of procinfo_static	5
Figure 09: Dynamic Compilation and Size of procinfo_dynamic	6
Figure 10: Size Check of Two Files	6
Figure 11: ELF Header Output (readelf -h)	7
Figure 12: Section to Segment Mapping (readelf -l)	8
Figure 13: Dynamic Section Output (readelf -d)	9
Figure 14: ldd Output for Dynamic and Static Binaries	9
Figure 15: ldd Output for Static Binary	9

LIST OF TABLES

Table I: Location Comparison of .h, .a and .so	5
Table II: Size Comparison of procinfo_static and procinfo_dynamic	6

INTRODUCTION

This is the lab report of Operating Systems, Security and Networks where we check how a programmer writes a C file that uses library functions like **printf()**, **getpid()**, and **time()**. When they compile it, many things in the background happen. The compiler changes the human-readable code to machine instructions. The linker then connects the program to external libraries. Finally, the OS reads the final binary file and loads it into the virtual memory ([Bryant & O'Hallaron, 2015](#)).

This lab is done to answer how a C program is linked to the C standard library, where does the library headers and compiled library code live on the disk, what is the difference between static and dynamic linking and how can we inspect the resulting executable files.

Sudo mkdir <repository name> //For creating a new folder.

Sudo touch<file name> //To create a new file

(Sudo is used to access files that regular user account doesn't have access to)

ls //list all files in the directory

gcc //to compile the program

readelf //to read the internal information of an ELF file

ldd //to find the shared libraries a program need

These are the basic linux commands used in the lab. But before doing that, we make a new folder to work on named 'LAB 2.1'

A terminal window screenshot showing a user named 'wagleayden' at a machine named 'Ayden'. The user is in the directory '~/os-labs/LAB 2.1'. They run 'ls' and see a list of folders: 'LAB 01', 'LAB 02', 'LAB 03', and 'LAB 04'. Then they run 'sudo mkdir 'LAB 2.1'', which prompts for a password. After authentication, they run 'ls' again and see the new folder 'LAB 2.1' added to the list. Finally, they run 'cd 'LAB 2.1'', changing into the newly created directory.

```
wagleayden@Ayden: ~/os-labs/LAB 2.1
wagleayden@Ayden:~/os-labs$ ls
'LAB 01'  'LAB 02'  'LAB 03'  'LAB 04'
wagleayden@Ayden:~/os-labs$ sudo mkdir 'LAB 2.1'
[sudo: authenticate] Password:
wagleayden@Ayden:~/os-labs$ ls
'LAB 01'  'LAB 02'  'LAB 03'  'LAB 04'  'LAB 2.1'
wagleayden@Ayden:~/os-labs$ cd 'LAB 2.1'
```

Figure 01: Make Lab2.1 Folder to Work With

A PRACTICAL C PROGRAM: PROCESS INFORMATION

A. Create procinfo.c

Here, a new file called '*procinfo.c*' was made. A C program was written that shows information about its own process. The program uses several functions from the C Standard Library:

- **getpid()** and **getppid()** from <unistd.h> gets the process ID and the parent process ID.
- **time()** and **localtime()** from <time.h> gets the current time.
- **printf()** and **asctime()** from <stdio.h> prints and formats the output.

Sudo nano <filename> //To open the file in nano editor to write or edit

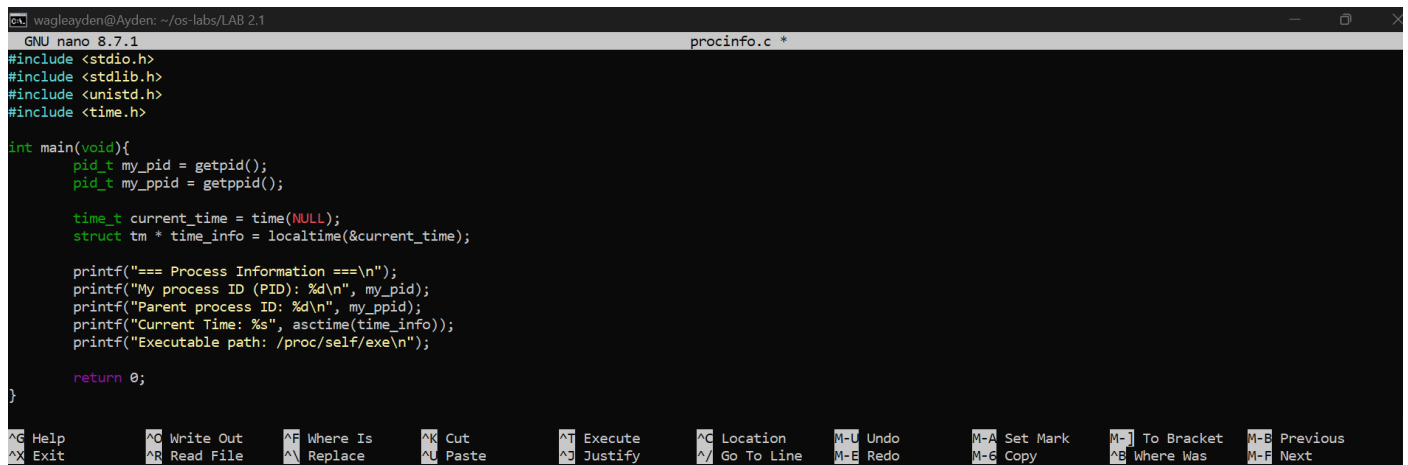
It is saved using **Ctrl + X, Y** and **Enter**

cat //To view the file without opening the editor

All of the code functions and the linux commands are implemented here before we compile and check the output as shown in figure below:

```
wagleayden@Ayden:~/os-labs$ ls
'LAB 01' 'LAB 02' 'LAB 03' 'LAB 04' 'LAB 2.1'
wagleayden@Ayden:~/os-labs$ cd 'LAB 2.1'
wagleayden@Ayden:~/os-labs/LAB 2.1$ ls
wagleayden@Ayden:~/os-labs/LAB 2.1$ sudo touch procinfo.c
wagleayden@Ayden:~/os-labs/LAB 2.1$ ls
procinfo.c
wagleayden@Ayden:~/os-labs/LAB 2.1$ sudo nano procinfo.c
wagleayden@Ayden:~/os-labs/LAB 2.1$
```

Figure 02: Make *procinfo.c* File



```
GNU nano 8.7.1 procinfo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

int main(void){
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    time_t current_time = time(NULL);
    struct tm * time_info = localtime(&current_time);

    printf("=== Process Information ===\n");
    printf("My process ID (PID): %d\n", my_pid);
    printf("Parent process ID: %d\n", my_ppid);
    printf("Current Time: %s", asctime(time_info));
    printf("Executable path: /proc/self/exe\n");

    return 0;
}
```

Figure 03: *procinfo.c* File in Nano Editor

```
wagleayden@Ayden:~/os-labs/LAB 2.1$ cat procinfo.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

int main(void){
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    time_t current_time = time(NULL);
    struct tm * time_info = localtime(&current_time);

    printf("=== Process Information ===\n");
    printf("My process ID (PID): %d\n", my_pid);
    printf("Parent process ID: %d\n", my_ppid);
    printf("Current Time: %s", asctime(time_info));
    printf("Executable path: /proc/self/exe\n");

    return 0;
}
```

Figure 04: procinfo.c File View using cat Command

B. Compile procinfo.c

```
sudo gcc hello.c -o <executable binary file> //Compiles the source code into a runnable program
./<executable binary file> //Runs the binary file in the terminal
```



```
wagleayden@Ayden: ~/os-labs/LAB 2.1
wagleayden@Ayden:~/os-labs/LAB 2.1$ sudo gcc procinfo.c -o procinfo
wagleayden@Ayden:~/os-labs/LAB 2.1$ ls
procinfo  procinfo.c
wagleayden@Ayden:~/os-labs/LAB 2.1$ ./procinfo
=== Process Information ===
My process ID (PID): 937
Parent process ID: 331
Current Time: Fri Oct 2 11:55:59 2026
Executable path: /proc/self/exe
wagleayden@Ayden:~/os-labs/LAB 2.1$
```

Figure 05: Run GCC compilation with result

The output shows the process ID, the parent process ID, and the current time. The next section answers where does the machine code for these functions come from.

THE C STANDARD LIBRARY: TWO HALVES

Every C program depends on the C Standard Library, called **libc**. On Linux, the GNU version is called **glibc**. The library is split into two physical parts: header files and precompiled libraries.

A. Part 1: Header Files: The Construction Manual

A header file is like a construction manual. They are stored in **/usr/include/** and have the extension **.h**. As mentioned, They contain declarations like function prototypes, macro definitions, type definitions (**typedef**) and structure definitions. They also tell the compiler what functions exist and how to call them. They do not contain any executable machine code ([Stevens & Rago, 2013](#)) .

```
ls /usr/include/stdio.h /usr/include/unistd.h //Checks that the header files exist in /usr/include/
grep "printf" /usr/include/stdio.h | head -2 //Searches for "printf" in stdio.h and shows the first 2 lines
grep "getpid" /usr/include/unistd.h | head -2 //Searches for "getpid" in unistd.h and shows the first 2
lines
grep -n "extern int printf" /usr/include/stdio.h //Finds the exact printf declaration and shows its line no.
```



```
wagleayden@Ayden: ~/os-labs
wagleayden@Ayden:~/os-labs$ ls /usr/include/stdio.h /usr/include/unistd.h
/usr/include/stdio.h  /usr/include/unistd.h
wagleayden@Ayden:~/os-labs$ grep "printf" /usr/include/stdio.h | head -2
/* Maximum length of printf output for a NaN. */
extern int fprintf(FILE * __restrict __stream,
wagleayden@Ayden:~/os-labs$ grep "getpid" /usr/include/unistd.h | head -2
extern __pid_t getpid(void) __THROW;
wagleayden@Ayden:~/os-labs$
```

Figure 06: Header File Check Using ls and grep

The **ls** output confirms that **stdio.h** and **unistd.h** are stored in **/usr/include/**.

The **grep** output shows only declarations. For example, `extern __pid_t getpid(void) __THROW;` tells the compiler that **getpid()** takes no arguments and returns a process ID, but it does not contain the real code. The **printf** search also returned the declaration of **fprintf**, which is another function of the same family. These files contain no machine code.

When `#include <stdio.h>` is written, the preprocessor copies the file into the source code before compilation. The compiler then knows these functions exist, but it does not know where their real code is stored yet.

B. Part 2: Precompiled Machine Code: The Actual Libraries

The real machine instructions for functions like **printf()**, **getpid()**, **time()** and **malloc()** are stored in library files. There are two types:

- **Static libraries (.a):** The machine instructions are copied directly into your program at compile time.
- **Shared (dynamic) libraries (.so):** the program just links to the library at runtime, so the code stays outside and is reused.

They are stored in locations like `/lib/x86_64-linux-gnu/`, `/usr/lib/x86_64-linux-gnu/`, `/usr/lib/` and `/lib/`

```
ls /lib/x86_64-linux-gnu/libc.so* //Lists the shared libc files
(the * matches every file starting with libc.so)
ls /usr/lib/x86_64-linux-gnu/libc.a //Checks that the static libc library (libc.a) exists
```



```
wagleayden@Ayden: ~/os-labs
wagleayden@Ayden:~/os-labs$ ls /lib/x86_64-linux-gnu/libc.so*
/lib/x86_64-linux-gnu/libc.so  /lib/x86_64-linux-gnu/libc.so.6
wagleayden@Ayden:~/os-labs$ ls /usr/lib/x86_64-linux-gnu/libc.a
/usr/lib/x86_64-linux-gnu/libc.a
wagleayden@Ayden:~/os-labs$
```

Figure 07: Checking Shared and Static libc Files

The first command shows that the shared C library is stored in `/lib/x86_64-linux-gnu/`. The file **libc.so.6** is the real shared library that programs load at run time.

The second command shows that the static library **libc.a** is stored in `/usr/lib/x86_64-linux-gnu/`. The real machine code of the C library is in these library files and not in the header files and this shows us exactly that. Header files only declare the functions, and the **.so** and **.a** files contain the compiled code.

C. Summary: The Two-Part Anatomy

The C Standard Library has two physical parts that do different jobs. The header files tell the compiler what functions exist. The precompiled libraries contain the real machine code of those functions.

The table below shows the location we guessed and the location that were shown in this system.

Table I: Location Comparison of .h, .a and .so

Component	Description	Predicted Location	Actual Location
Headers (.h)	Declaration, prototypes, macros	/usr/include/	/usr/include/
Static lib (.a)	Precompiled object code (archive)	/usr/lib/	/usr/lib/x86_64-linux-gnu/
Shared lib (.so)	Precompiled position-independent code	/lib/	/lib/x86_64-linux-gnu/

The files found match this table.

STATIC VS. DYNAMIC LINKING

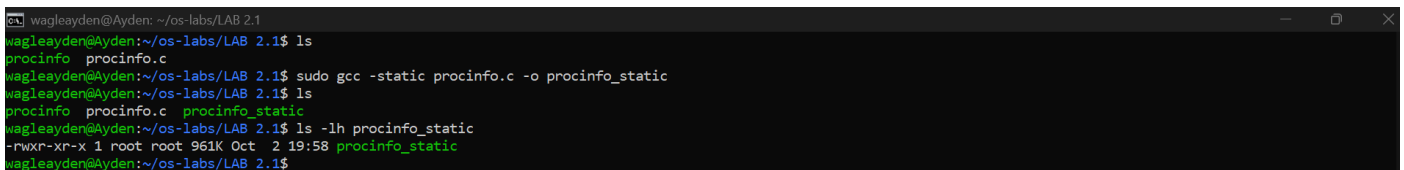
Linking is the process of joining your program with library code so all the pieces work together as one executable. Compiling turns source code into object files and linking connects those files with libraries whether they are static or dynamic to make the final runnable program.

Static linking means the library's code is copied into the program at compile time, while dynamic linking means the program connects to external library code at runtime ([Levine, 2000](#)) .

A. Static Linking

```
gcc -static procinfo.c -o procinfo_static // Compiles with static linking
(libc code is copied into the file)
```

```
ls -lh procinfo_static // Shows file sizes in human-readable form
(-l long format, -h K/M units)
```



```
wagleayden@Ayden: ~/os-labs/LAB 2.1
wagleayden@Ayden:~/os-labs/LAB 2.1$ ls
procinfo  procinfo.c
wagleayden@Ayden:~/os-labs/LAB 2.1$ sudo gcc -static procinfo.c -o procinfo_static
wagleayden@Ayden:~/os-labs/LAB 2.1$ ls
procinfo  procinfo.c  procinfo_static
wagleayden@Ayden:~/os-labs/LAB 2.1$ ls -lh procinfo_static
-rwxr-xr-x 1 root root 961K Oct  2 19:58 procinfo_static
wagleayden@Ayden:~/os-labs/LAB 2.1$
```

Figure 08: Static Compilation and Size of procinfo_static

Procinfo.c was compiled with the **-static** option. The output file *procinfo_static* is 961K. It is large because it contains the libc code copied from **libc.a**.

B. Dynamic Linking

```
gcc -static procinfo.c -o procinfo_dynamic // Compiles with dynamic linking
(the default, uses libc.so.6 at run time)
```

```
ls -lh procinfo_dynamic // Shows the size of the dynamic file
```

```
wagleayden@Ayden: ~/os-labs/LAB 2.1
wagleayden@Ayden:~/os-labs/LAB 2.1$ ls
procinfo  procinfo.c  procinfo_static
wagleayden@Ayden:~/os-labs/LAB 2.1$ sudo gcc procinfo.c -o procinfo_dynamic
wagleayden@Ayden:~/os-labs/LAB 2.1$ ls
procinfo  procinfo.c  procinfo_dynamic  procinfo_static
wagleayden@Ayden:~/os-labs/LAB 2.1$ ls -lh procinfo_dynamic
-rwxr-xr-x 1 root root 16K Oct  2 20:07 procinfo_dynamic
wagleayden@Ayden:~/os-labs/LAB 2.1$
```

Figure 09: Dynamic Compilation and Size of `procinfo_dynamic`

The dynamic binary `procinfo_dynamic` is only 16K. It is small because it stores only references to `libc.so.6`.

C. Why It Matters

```
ls -lh procinfo_static procinfo_dynamic // Shows both sizes in human-readable form
(-l long format, -h K/M units)
```

```
wagleayden@Ayden: ~/os-labs/LAB 2.1
wagleayden@Ayden:~/os-labs/LAB 2.1$ ls -lh procinfo_static procinfo_dynamic
-rwxr-xr-x 1 root root 16K Oct  2 20:07 procinfo_dynamic
-rwxr-xr-x 1 root root 961K Oct  2 19:58 procinfo_static
wagleayden@Ayden:~/os-labs/LAB 2.1$
```

Figure 10: Size Check of Two Files

Table II: Size Comparison of `procinfo_static` and `procinfo_dynamic`

Binary	Linking Type	Size
<code>procinfo_static</code>	Static	961K
<code>procinfo_dynamic</code>	Dynamic	16K

A static binary is much larger than a dynamic one. This is because it stores all the library code inside itself. A dynamic binary is smaller because it loads the code from shared libraries like `libc.so.6` when it runs. Dynamic linking saves memory and disk space across the system. Many programs share one copy of the library. But it also creates runtime dependencies and possible version conflicts. Static linking avoids these problems. The program carries everything it needs. However, the binaries are bigger and they don't get automatic fixes when the library is updated ([Levine, 2000](#)).

INSPECTING EXECUTABLES WITH **READELF**

On Linux, compiled programs use the ELF (Executable and Linkable Format). The `readelf` tool lets us look inside an ELF file without running it.

A. Three Views of an Executable

ELF Header (`readelf -h`): Basic info about the file, like whether it's valid and which CPU it runs on.

Program Headers (`readelf -l`): Instructions for the operating system loader: which parts of the file go into memory and with what permissions.

Dynamic Section (readelf -d): Info for the dynamic linker: which shared libraries need to be loaded.

B. View 1: The ELF Header (readelf -h)

```
readelf -h procinfo_dynamic // Reads the ELF header of the dynamic program
```

```
wagleayden@Ayden: ~/os-labs/LAB 2.1
wagleayden@Ayden:~/os-labs/LAB 2.1$ readelf -h procinfo_dynamic
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF64
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  DYN (Position-Independent Executable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x1140
  Start of program headers:              64 (bytes into file)
  Start of section headers:             14264 (bytes into file)
  Flags:                                  0x0
  Size of this header:                   64 (bytes)
  Size of program headers:               56 (bytes)
  Number of program headers:             14
  Size of section headers:               64 (bytes)
  Number of section headers:             31
  Section header string table index:     30
```

Figure 11: ELF Header Output (readelf -h)

On my system, the Magic bytes were **7f 45 4c 46**, so the file is a valid ELF file.

The Class was ELF64 (64-bit) and the Data was 2's complement, little endian. The Type was DYN (Position-Independent Executable file), which is normal for modern dynamically linked programs. The Machine was Advanced Micro Devices X86-64 and the entry point address was 0x1140. The file has 14 program headers and 31 section headers

C. View 2: The Program Headers (readelf -l)

```
readelf -l procinfo_dynamic // Reads the program headers (segments) of the dynamic program
```

```
wagleayden@Ayden: ~/os-labs/LAB 2.1
wagleayden@Ayden:~/os-labs/LAB 2.1$ readelf -l procinfo_dynamic
Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1140
There are 14 program headers, starting at offset 64

Program Headers:
Type           Offset             VirtAddr           PhysAddr
               FileSiz            MemSiz             Flags  Align
PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
               0x0000000000000310 0x0000000000000310 R      0x8
INTERP         0x0000000000000374 0x0000000000000374 0x0000000000000374
               0x00000000000001c 0x00000000000001c R      0x1
               [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000
               0x00000000000007b8 0x00000000000007b8 R      0x1000
LOAD           0x0000000000001000 0x0000000000001000 0x0000000000001000
               0x0000000000000311 0x0000000000000311 R E    0x1000
LOAD           0x0000000000002000 0x0000000000002000 0x0000000000002000
               0x0000000000001c0 0x0000000000001c0 R      0x1000
LOAD           0x0000000000002d80 0x0000000000003d80 0x0000000000003d80
               0x000000000000290 0x000000000000298 RW    0x1000
DYNAMIC        0x0000000000002d90 0x0000000000003d90 0x0000000000003d90
               0x0000000000001f0 0x0000000000001f0 RW      0x8
NOTE           0x0000000000000350 0x0000000000000350 0x0000000000000350
               0x0000000000000024 0x0000000000000024 R      0x4
NOTE           0x0000000000002170 0x0000000000002170 0x0000000000002170
               0x0000000000000030 0x0000000000000030 R      0x8
NOTE           0x00000000000021a0 0x00000000000021a0 0x00000000000021a0
               0x0000000000000020 0x0000000000000020 R      0x4
GNU_PROPERTY   0x0000000000002170 0x0000000000002170 0x0000000000002170
               0x0000000000000030 0x0000000000000030 R      0x8
GNU_EH_FRAME   0x0000000000002088 0x0000000000002088 0x0000000000002088
               0x0000000000000034 0x0000000000000034 R      0x4
GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
               0x0000000000000000 0x0000000000000000 RW    0x10
GNU_RELRO      0x0000000000002d80 0x0000000000003d80 0x0000000000003d80
               0x000000000000280 0x000000000000280 R      0x1
```

```

Segment Sections...
00
01 .interp
02 .note.gnu.build-id .interp .gnu.hash .dynsym .dynstr .gnu.version .gnu.version_r .rela.dyn .rela.plt
03 .init .plt .plt.got .plt.sec .text .fini
04 .rodata .eh_frame_hdr .eh_frame .note.gnu.property .note.ABI-tag
05 .init_array .fini_array .dynamic .got .data .bss
06 .dynamic
07 .note.gnu.build-id
08 .note.gnu.property
09 .note.ABI-tag
10 .note.gnu.property
11 .eh_frame_hdr
12
13 .init_array .fini_array .dynamic .got

```

Figure 12: Section to Segment Mapping (readelf -l)

INTERP: The program interpreter was `/lib64/ld-linux-x86-64.so.2`. This is the dynamic linker that the kernel loads before `main()` runs.

LOAD: There were 4 LOAD segments:

1. **R (offset 0x0, size 0x7b8):** read-only data and the dynamic linking information.
2. **R E (offset 0x1000, size 0x311):** executable code. The mapping shows it holds `.init`, `.plt`, `.text` and `.fini`.
3. **R (offset 0x2000, size 0x1c0):** read-only data such as `.rodata`.
4. **RW (offset 0x2d80, size 0x290):** read-write data. It holds `.data`, `.bss`, `.dynamic` and `.got`.

GNU_STACK: The flags were RW, so the stack is readable and writable but not executable. This is the secure setting.

There is also a `GNU_RELRO` segment (flags R). It makes some parts of the file like `.got` and `.dynamic` read-only after the program is loaded. This is a security feature ([Tool Interface Standard Committee, 1995](#)).

D. View 3: The Dynamic Section (readelf -d)

`readelf -d procinfo_dynamic` // Reads the dynamic section of the dynamic program

```

wagleayden@Ayden: ~/os-labs/LAB 2.1
wagleayden@Ayden:~/os-labs/LAB 2.1$ readelf -d procinfo_dynamic

Dynamic section at offset 0x2d90 contains 27 entries:
  Tag             Type              Name/Value
 0x0000000000000001 (NEEDED)          Shared library: [libc.so.6]
 0x000000000000000c (INIT)            0x1000
 0x000000000000000d (FINI)            0x1304
 0x0000000000000019 (INIT_ARRAY)      0x3d80
 0x000000000000001b (INIT_ARRAYSZ)      8 (bytes)
 0x000000000000001a (FINI_ARRAY)      0x3d88
 0x000000000000001c (FINI_ARRAYSZ)      8 (bytes)
 0x00000000000000ff (GNU_HASH)          0x390
 0x0000000000000005 (STRTAB)            0x508
 0x0000000000000006 (SYMTAB)            0x3b8
 0x000000000000000a (STRSZ)              208 (bytes)
 0x000000000000000b (SYMENT)             24 (bytes)
 0x0000000000000015 (DEBUG)              0x0
 0x0000000000000003 (PLTGOT)             0x3f80
 0x0000000000000002 (PLTRELSZ)           192 (bytes)
 0x0000000000000014 (PLTREL)             RELA
 0x0000000000000017 (JMPREL)             0x6f8
 0x0000000000000007 (RELA)               0x638
 0x0000000000000008 (RELA_SZ)            192 (bytes)
 0x0000000000000009 (RELAENT)            24 (bytes)
 0x000000000000001e (FLAGS)              BIND_NOW
 0x00000000000000ff (FLAGS_1)            Flags: NOW PIE
 0x00000000000000fe (VERNEED)            0x5f8
 0x00000000000000ff (VERNEEDNUM)         1
 0x0000000000000100 (VERSYM)             0x5d8
 0x0000000000000101 (RELACOUNT)          3
 0x0000000000000000 (NULL)              0x0
wagleayden@Ayden:~/os-labs/LAB 2.1$

```

Figure 13: Dynamic Section Output (readelf -d)

TRACING DEPENDENCIES WITH **ldd**

The **ldd** command in Linux shows the shared libraries that an executable depends on.

ldd procinfo_dynamic // Shows the shared libraries the dynamic program needs and where they are
ldd procinfo_static // Checks if the static program has any shared library dependencies

```
wagleayden@Ayden: ~/os-labs/LAB 2.1
wagleayden@Ayden:~/os-labs/LAB 2.1$ ldd procinfo_static
not a dynamic executable
wagleayden@Ayden:~/os-labs/LAB 2.1$
wagleayden@Ayden:~/os-labs/LAB 2.1$ ldd procinfo_dynamic
linux-vdso.so.1 (0x00007ea0d9c05000)
libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x00007ea0d9800000)
/lib64/ld-linux-x86-64.so.2 (0x00007ea0d9c07000)
wagleayden@Ayden:~/os-labs/LAB 2.1$
```

Figure 14: ldd Output for Dynamic and Static Binaries

ldd procinfo_dynamic showed three entries:

1. **linux-vdso.so.1**: the virtual library that the kernel adds to every process.
2. **libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6**: the real location of libc on my system.
3. **/lib64/ld-linux-x86-64.so.2**: the dynamic linker.

A. Static Binaries and **ldd**

```
wagleayden@Ayden: ~/os-labs/LAB 2.1
wagleayden@Ayden:~/os-labs/LAB 2.1$ ldd procinfo_static
not a dynamic executable
wagleayden@Ayden:~/os-labs/LAB 2.1$
wagleayden@Ayden:~/os-labs/LAB 2.1$ ldd procinfo_dynamic
linux-vdso.so.1 (0x00007ea0d9c05000)
libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x00007ea0d9800000)
/lib64/ld-linux-x86-64.so.2 (0x00007ea0d9c07000)
wagleayden@Ayden:~/os-labs/LAB 2.1$
```

Figure 15: ldd Output for Static Binary

ldd procinfo_static printed "not a dynamic executable". This means the static binary has no shared library dependencies ([Levine, 2000](#)) .

KNOWLEDGE CHECK QUESTIONS

Question 1: What is the fundamental difference between a C header file and a shared library?

A header file (.h) has only declarations but no machine code. The compiler uses it at compile time.

On my system, `stdio.h` is in `/usr/include/`.

A shared library (.so) has the real compiled machine code. The dynamic linker loads it when the program runs, and many programs share one copy.

On my system, `libc.so.6` is in `/lib/x86_64-linux-gnu/`.

Question 2: What is static and dynamic linking?

Static linking is when the library code is copied into the program at compile time, making the executable self-contained but larger. the linker copies the needed code from `.a` files into the executable at compile time. `procinfo_static` was 961K and needed no shared libraries.

Dynamic linking is when the program uses shared libraries at runtime, keeping the executable smaller but dependent on external files. The executable stores only references to `.so` files, and the dynamic linker loads them at run time. `procinfo_dynamic` was 16K and needed `libc.so.6`.

Question 3: What is the ELF Header? Write down the commands to check headers.

The ELF Header is the first part of an ELF file.

It stores the magic number, class (32/64-bit), endianness, file type, CPU type (Machine) and entry point address. The OS uses it to check that the file is a valid executable for this CPU.

On this system, ELF64, little endian, DYN (PIE), x86-64, entry point 0x1140 is from ELF Header.

ELF Header: `readelf -h procinfo_dynamic`

Program Headers: `readelf -l procinfo_dynamic`

Dynamic Section: `readelf -d procinfo_dynamic`

Shared Library Paths: `ldd procinfo_dynamic`

Question 4: What does bookmark mean? Provide an example.

A bookmark is a saved marker that points to a specific location so you can jump back to it quickly without searching again. I can reference [Table 1](#) using a bookmark which takes me to the table.

CONCLUSION

Library functions like `printf()`, `getpid()` and `time()` were used in C program *procinfo.c*. C Standard Library has two parts that consists of header files in `/usr/include/` which only declare functions and another is precompiled libraries like `libc.so.6` and `libc.a` in `/lib/x86_64-linux-gnu/` and `/usr/lib/x86_64-linux-gnu/`. which contain the real machine code.

Then the program was compiled with static and dynamic linking. The static file was much bigger because it contains the libc code. The dynamic file loads `libc.so.6` at run time.

With `readelf`, it was noticed that the dynamic file is a 64-bit DYN (PIE) file for x86-64, with INTERP `/lib64/ld-linux-x86-64.so.2`, a secure RW stack, and `libc.so.6` as the NEEDED library. With `ldd`, it can be seen that libc stays at `/usr/lib/x86_64-linux-gnu/libc.so.6`, and the static file gives "not a dynamic executable".

The static file also had no INTERP entry. This lab helped me understand how a program is linked and loaded by the operating system.

GITHUB REPOSITORY

[GitHub Repository](#)

REFERENCES

- Bryant, R. E., & O'Hallaron, D. R. (2015). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.
- Levine, J. R. (2000). *Linkers and loaders*. Morgan Kaufmann.
- Stevens, W. R., & Rago, S. A. (2013). *Advanced programming in the UNIX environment* (3rd ed.). Addison-Wesley.
- Tool Interface Standard Committee. (1995). *Tool Interface Standard (TIS) executable and linking format (ELF) specification* (Version 1.2).